
Edge AI on Raspberry Pi 5

Real-Time Object Detection with LiteRT and YOLO

An implementation and performance report — from OS setup and the LiteRT CLI to a live camera detector recording annotated video, with a full system resource analysis under sustained inference load.

Platform	Raspberry Pi 5 (8 GB) · Raspberry Pi OS 64-bit (aarch64)
Camera	Camera Module 3 (Sony IMX708, CSI)
Runtime	LiteRT CLI · Ultralytics 8.4.x · YOLO11n
Date	16 August 2026
Status	Camera, classification, detection & recording operational

Contents

1	Objective & Scope
2	System Setup — OS, Access & LiteRT CLI
3	Baseline Inference — Image Classification
4	Object Detection — Model Selection & Pitfalls
5	Live Camera Detector with Video Recording
6	Environment & Colour-Space Fixes
7	Performance Analysis Under Load
8	Findings, Conclusions & Next Steps
A	Command & Artifact Reference

1 Objective & Scope

This report documents the end-to-end deployment of on-device ("edge") AI on a Raspberry Pi 5, following the workflow published by Raspberry Pi and Google around **LiteRT** (the runtime formerly known as TensorFlow Lite). The practical goal was to progress from a fresh OS install to a working **real-time object detector** that draws bounding boxes on a live camera feed and records the annotated video to disk — while keeping execution on the LiteRT-native stack. A secondary objective was to instrument the system and characterise its behaviour under sustained inference load.

Work completed

- Provisioned Raspberry Pi OS (64-bit) and installed the LiteRT CLI via the uv environment manager.
- Validated the runtime with a language model (Gemma) and an image-classification model (EfficientNet-B1).
- Diagnosed a failed detection-model download and pivoted to an Ultralytics-exported YOLO model for a guaranteed-valid artifact.
- Built a live camera detector (CSI / IMX708) with automatic box decoding and annotated MP4 recording.
- Resolved a NumPy 1.x/2.x binary-incompatibility and a camera colour-space error.
- Captured and analysed a 120-second CPU / GPU / thermal / memory profile under load.

2 System Setup — OS, Access & LiteRT CLI

The Pi was imaged with Raspberry Pi OS (64-bit) using Raspberry Pi Imager, with SSH enabled and a hostname/username configured at flash time. After connecting over the network, the architecture was confirmed as aarch64 — a prerequisite for LiteRT.

```
ping <hostname>.local      # confirm reachable
ssh <user>@<hostname>.local # remote login
uname -m                   # -> aarch64 (64-bit confirmed)
```

The LiteRT CLI was installed inside an isolated Python 3.13 environment created by the uv manager. The CLI aggregates model download, inference and benchmarking behind a single command set.

```
sudo apt update && sudo apt full-upgrade -y
curl -LsSf https://astral.sh/uv/install.sh | sh
source $HOME/.local/bin/env
uv venv --clear --python=3.13 --seed
source .venv/bin/activate
uv pip install litert-cli-nightly
```

NOTE. A Hugging Face access token (Read scope) is required for gated model repositories and is supplied via the HUGGING_FACE_HUB_TOKEN environment variable before running any model that pulls from the Hub.

3 Baseline Inference — Image Classification

Before attempting detection, the runtime was validated with EfficientNet-B1, an ImageNet image classifier. Stills were captured from the CSI camera with `rpicam-still` and classified through the LiteRT CLI.

```
litert download litert-community/efficientnet_b1 --output efficientnet
rpicam-still -o capture.jpg
litert run efficientnet/efficientnet_b1.tflite --input capture.jpg
```

Representative outputs

Inference completed in roughly 90–115 ms per image. The model input signature was `[1, 3, 240, 240]` (channels-first), a detail that later informed preprocessing decisions. Two confident results are shown below.

Scene	Top-1 prediction	Score	Runner-up	Score
Desk	desk	7.74	desktop computer	6.08
Screen	web site	7.20	monitor	5.57

FINDING. A wide gap between the top-1 and top-2 scores indicates a confident classification. Early captures showed all five scores bunched near 4.5–5.4 — the model was uncertain because the camera was aimed at a general scene rather than one clean object. A whole-frame classifier forces a single label on the entire image; this motivated the move to object *detection*, which localises and labels multiple objects independently.

NOTE. `litert run` is only ever as fresh as the last `rpicam-still`. Re-running inference without re-capturing returns an identical result on the same file — observed when three consecutive runs produced byte-identical output.

4 Object Detection — Model Selection & Pitfalls

The initial attempt to download a ready-made detection model failed, and the failure cascaded into a second, misleading error:

```
litert download litert-community/yolo26n --output yolo
-> 404 Client Error. Repository Not Found (litert-community/yolo26n)
litert run yolo/yolo26n.tflite --input capture.jpg
-> RuntimeError: Failed to build flatbuffer model
```

PITFALL. The repository id did not exist, so nothing downloaded; the subsequent "Failed to build flatbuffer model" was simply the runtime trying to load an absent / empty file. Root cause was a wrong model id, **not** a code or runtime fault.

Rather than continue guessing repository names, the project pivoted to **Ultralytics** — the toolkit credited in the source article — which can export a YOLO model directly to a valid `.tflite` that still executes on the LiteRT/TFLite runtime. This removes two unknowns at once: it guarantees a valid model artifact, and it yields a documented, predictable output format.

```
pip install ultralytics
yolo export model=yolo11n.pt format=tflite imgsz=640
# -> yolo11n_saved_model/yolo11n_float32.tflite (valid flatbuffer)
```

A key advantage of the Ultralytics path is that box decoding, non-max suppression (NMS) and label rendering are handled internally via `results[0].plot()`, eliminating the fragile manual post-processing that a raw-tensor approach would require.

5 Live Camera Detector with Video Recording

The CSI camera (IMX708) is driven through `picamera2` rather than an OpenCV `source=0` device, because CSI cameras route through `libcamera`, not `V4L2`. Each captured frame is passed to the model; the annotated result is both displayed and written to an MP4 via `cv2.VideoWriter`. The core loop is shown below.

```
from picamera2 import Picamera2
from ultralytics import YOLO
import cv2, time

model = YOLO("yolo11n.pt") # or the exported .tflite for LiteRT-native
picam2 = Picamera2()
picam2.configure(picam2.create_preview_configuration(
    main={"format": "RGB888", "size": (640, 640)}))
picam2.start(); time.sleep(1)

fourcc = cv2.VideoWriter_fourcc(*"mp4v")
writer = cv2.VideoWriter("detections.mp4", fourcc, 20.0, (640, 640))

while True:
    frame = picam2.capture_array()
    results = model.predict(frame, verbose=False)
    annotated = results[0].plot() # boxes + labels baked in
    writer.write(annotated) # record annotated frame
    cv2.imshow("YOLO", annotated)
    if cv2.waitKey(1) == ord("q"):
        break
writer.release(); picam2.stop(); cv2.destroyAllWindows()
```

PITFALL. `writer.release()` must run to finalise the MP4 footer. If the process is killed (Ctrl+C) instead of quitting with `q`, the file can be left corrupt. Recording FPS is a metadata stamp on the file, independent of the real loop rate — mismatches only affect playback speed.

6 Environment & Colour-Space Fixes

6.1 NumPy binary incompatibility

Installing Ultralytics pulled in NumPy 2.x, while the system-installed `picamera2` / `simplejpeg` were compiled against NumPy 1.x. The ABI mismatch broke the camera import:

```
ValueError: numpy.dtype size changed, may indicate binary
incompatibility. Expected 96 from C header, got 88 from PyObject
```

Resolution — pin NumPy to the 1.x series (and prefer a system-site-packages venv so the apt-built camera stack is visible):

```
pip install "numpy<2"
# robust alternative:
sudo apt install -y python3-picamera2
python3 -m venv --system-site-packages .venv
```

6.2 Camera colour channel order

picamera2's "RGB888" format returns array data in **BGR** channel order — the same convention Ultralytics, cv2.imshow and VideoWriter all expect. The pipeline is therefore consistently BGR end-to-end, and **no** colour conversion should be applied. An added cvtColor(RGB2BGR) introduced a single spurious swap and was removed to restore correct colour.

NOTE. If colours still invert on a given picamera2 build, switch the capture format from "RGB888" to "BGR888" to flip channel order at the source rather than converting per-frame.

7 Performance Analysis Under Load

A 120-second profile (226 samples, ~0.53 s interval) was captured while the detector ran continuously on the CPU. The traces below cover per-core CPU utilisation, the ARM and V3D clocks, and SoC temperature.

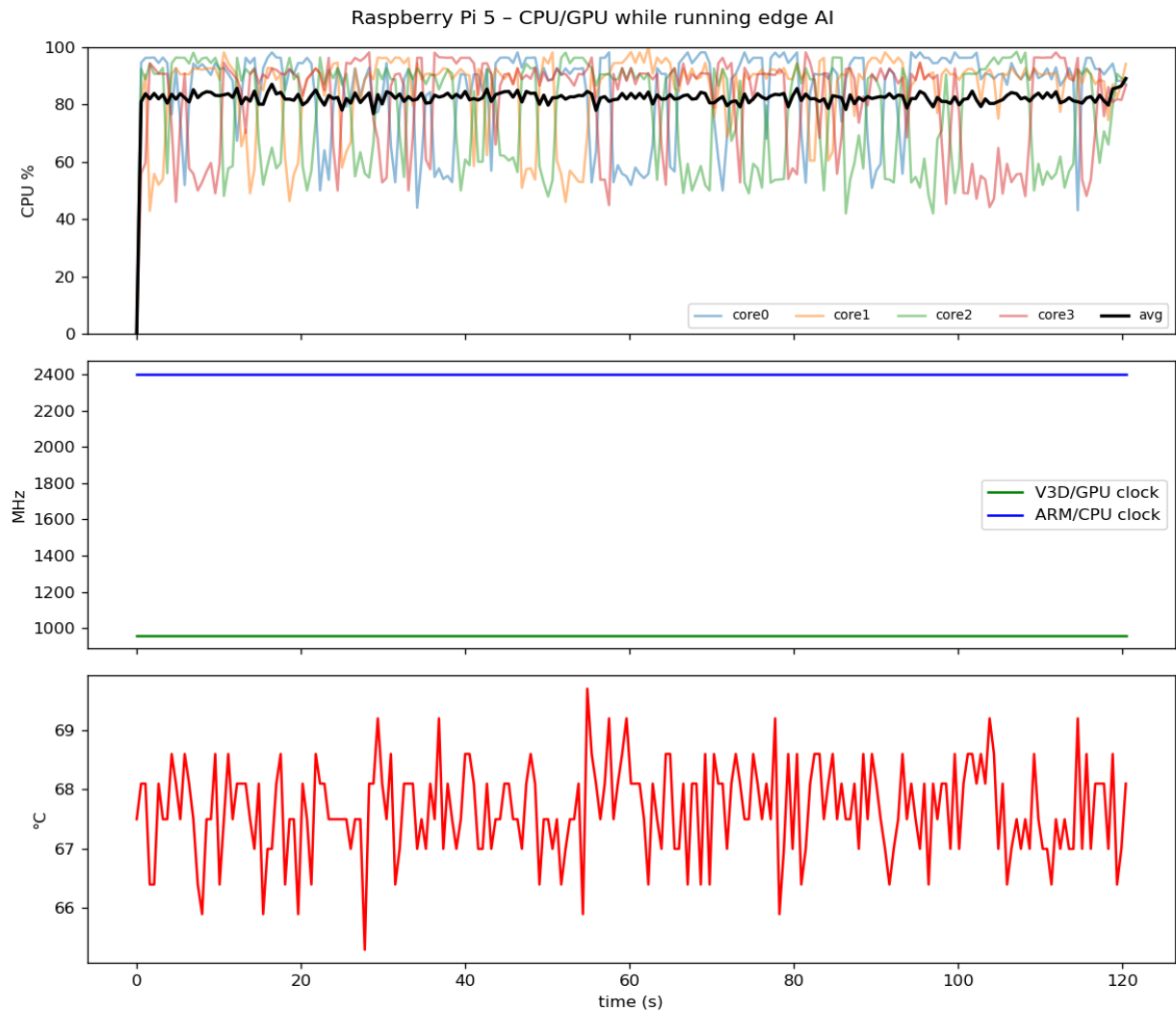


Figure 1 — CPU / GPU clock / thermal traces over a 120 s inference run.

Measured summary

Metric	Min	Mean	Max	Reading
CPU average (%)	76.7	82.3	89.0	Sustained high load
Core 0 / 1 / 2 / 3 mean (%)	42.0	—	100.0	All four cores active; work migrates across cores
Memory (%)	64.2	65.2	67.1	Flat — no leak, no swap pressure
SoC temp (°C)	65.3	67.7	69.7	Cool; far below ~80 °C throttle point
ARM / CPU clock (MHz)	2400	2400	2400	Pinned at max — zero throttling
V3D / GPU clock (MHz)	960	960	960	Idle floor — GPU unused (CPU path)

FINDING. The average CPU was at or above 80% for 93% of the run, all four cores engaged — the signature of multi-threaded LiteRT inference. The workload is **CPU-bound**.

FINDING. The V3D/GPU clock never left its 960 MHz idle floor, confirming execution stayed on the CPU. This is the correct choice: for a single model, the Pi 5 CPU outperforms the experimental WebGPU/Vulkan path. GPU offload pays off only when running a second model concurrently (e.g. detection on GPU while an LLM uses the CPU).

FINDING. Thermals gave ~10 °C of headroom and the ARM clock held a constant 2400 MHz with no throttling — there is spare capacity for larger models, higher resolution, or longer sessions without additional cooling.

8 Findings, Conclusions & Next Steps

Key findings

- **Pipeline operational.** Camera capture, classification, YOLO detection and annotated MP4 recording all function on the Pi 5.
- **CPU-bound, thermally comfortable.** Inference saturates the CPU (~82% avg) while the SoC stays near 68 °C with no clock throttling.
- **GPU idle by design.** The CPU path is faster for a single model; the GPU only earns its place under concurrent multi-model workloads.
- **Two environment traps resolved.** A NumPy 1.x/2.x ABI clash and a picamera2 BGR colour-order subtlety were the only real blockers.

Recommended next steps

- Export an **int8-quantized** .tflite to raise frame rate while remaining on the LiteRT runtime.
- Lower capture resolution (e.g. 480 px) as a fast accuracy-for-speed trade when needed.
- Log measured inference FPS alongside CPU load to quantify gains from quantization.
- Explore concurrent models (detection + language) to make deliberate use of the idle GPU, mirroring the reference robotics pipeline.
- Add Ctrl+C-safe finalisation (try/finally) and timestamped output filenames for robust unattended recording.

Appendix A Command & Artifact Reference

Purpose	Command / Artifact
Confirm architecture	<code>uname -m → aarch64</code>
Install LiteRT CLI	<code>uv pip install litert-cli-nightly</code>
Capture still	<code>rpicam-still -o capture.jpg</code>
Classify image	<code>litert run efficientnet/efficientnet_b1.tflite --input capture.jpg</code>
Export YOLO to TFLite	<code>yolo export model=yolo11n.pt format=tflite imgsz=640</code>
Fix NumPy ABI clash	<code>pip install "numpy<2"</code>
Live detector + recording	<code>monitor.py</code> (see Section 5)
Recorded output	<code>detections.mp4</code> (annotated H.264/MP4)

Prepared as a technical record of the LiteRT / YOLO edge-AI deployment on Raspberry Pi 5. Performance figures are derived directly from the captured monitoring dataset (226 samples over 120 seconds).